
CUDA programming and ML compilers

A step-by-step optimization journey

Li Shang

ML Systems

Li Shang

One computation, three views

01

Express the work

Describe the workload via CUDA programming without the need of considering GPU hardware details.

02

Move data efficiently

Move data efficiently across memory hierarchy to maximize computation density.

03

Automate the choices

How to automate the optimization process without human intervention.

01

CUDA execution

How does mathematical work become GPU threads?

Workloads that suit GPUs

Parallel work

Many output elements can be computed at the same time because they do not depend on one another.

Enough arithmetic

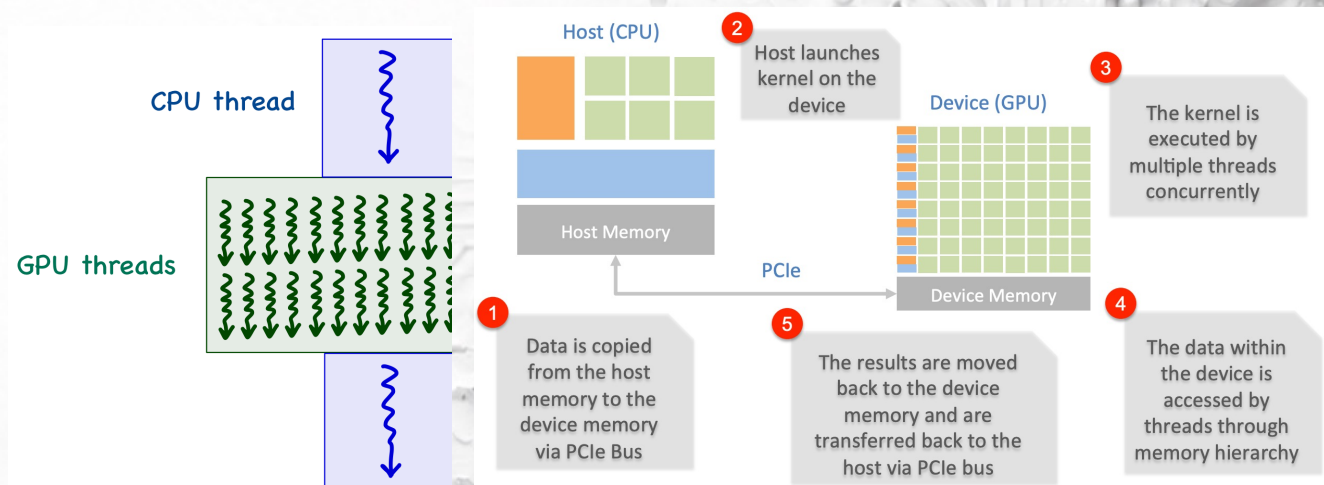
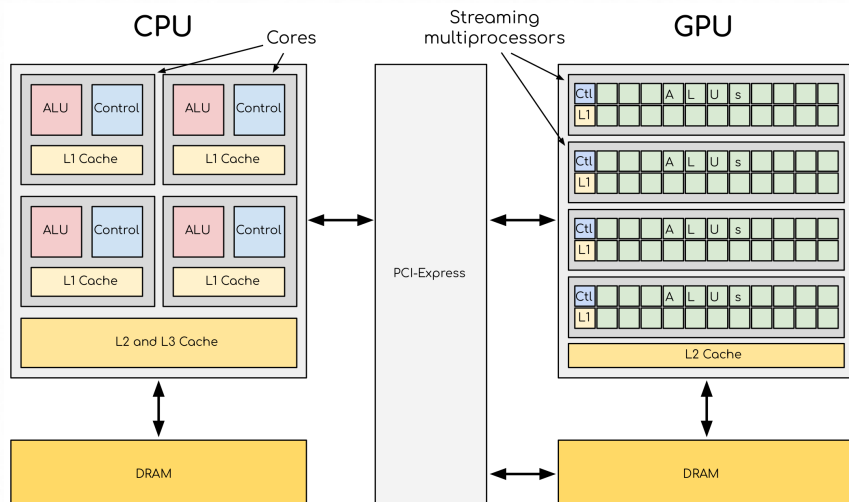
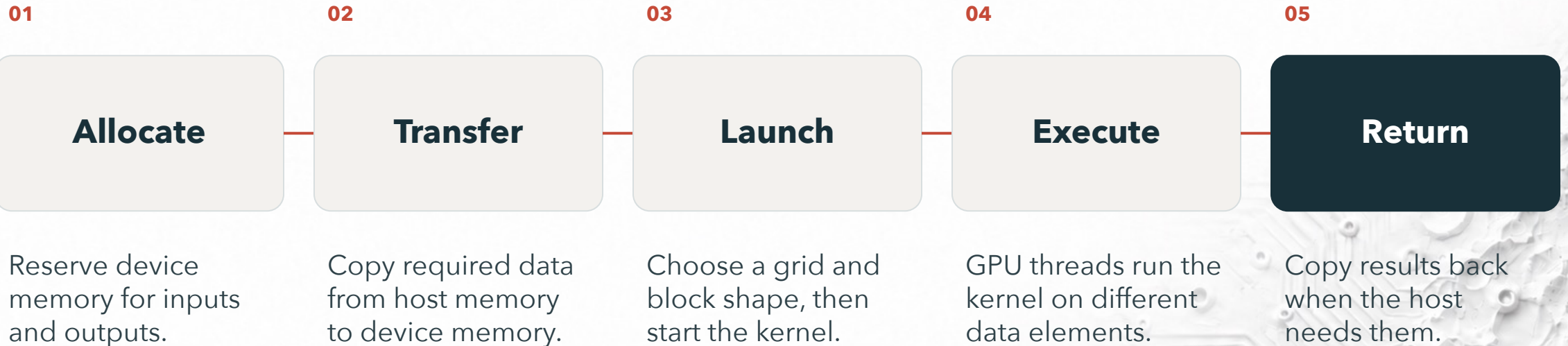
Each transferred value should support enough useful operations to amortize memory movement and launch overhead.

Regular structure

Predictable control flow and memory access help minimizes optimization complexity and uncertainty.

The conventional CUDA host-accelerator workflow

The CPU prepares work. The GPU executes a kernel across many threads.



The CPU prepares data and launches the kernel

1 Create CPU arrays

N = 65,536 elements

2 Allocate GPU arrays

Reserve A, B, and C.

3 Copy inputs to GPU

Copy both A and B.

4 Launch the kernel

T threads per block.

G blocks cover N elements.

5 Retrieve the result

The copy waits for the result in this default-stream example.

6 Release GPU arrays

CPU vectors clean up automatically.

```
const int N = 1 << 16;
const size_t bytes = N * sizeof(float);
std::vector<float> h_A(N, 1.0f), h_B(N, 2.0f), h_C(N);

float *d_A, *d_B, *d_C;
cudaMalloc(&d_A, bytes); cudaMalloc(&d_B, bytes);
cudaMalloc(&d_C, bytes);

cudaMemcpy(d_A, h_A.data(), bytes, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B.data(), bytes, cudaMemcpyHostToDevice);

const int T = 256;
const int G = (N + T - 1) / T;
vectorAdd<<<G, T>>>>(d_A, d_B, d_C, N);

cudaMemcpy(h_C.data(), d_C, bytes, cudaMemcpyDeviceToHost);
// Every h_C[i] should now be 3.0f.

cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
```

Vector addition: CPU loop and GPU kernel

CPU: one thread visits every element

```
for (int i = 0; i < N; ++i) {  
    C[i] = A[i] + B[i];  
}
```

The loop chooses the next index i .
Each iteration writes one result.

GPU: each thread computes one element

```
__global__ void vectorAdd(  
    const float* A, const float* B,  
    float* C, int N) {  
    int i = blockIdx.x * blockDim.x  
        + threadIdx.x;  
    if (i < N) C[i] = A[i] + B[i];  
}
```

`__global__` marks a GPU kernel that the CPU can launch.

THE PROGRAMMING MODEL

A kernel describes one thread's work

Launches many kernel instances. Each instance reads its thread/block indices, selects a data element, and performs the same program on that element.

- **Entry point for GPU computation:** The kernel defines what each thread does.
- **Single Program, Multiple Data (SPMD)** style: All threads run the same program (kernel), but each thread is distinguished by its thread/block ID.
- **Massively parallel:** Thousands of threads can be executing the kernel simultaneously.

THE PROGRAMMING MODEL

A kernel describes one thread's work

Launches many kernel instances. Each instance reads its thread/block indices, selects a data element, and performs the same program on that element.

How do we determine the mapping between threads and the data they operate on?

```
// HOST code to queue kernel  
simpleKernel <<< dimGrid, dimBlock >>> (N, d_a);
```

`__global__` specifies kernel

```
__global__ void simpleKernel(int N, float *d_a){
```

```
// Convert thread and thread-block indices into array index  
const int n = threadIdx.x + blockDim.x*blockIdx.x;
```

```
// If index is in [0,N-1] add entries  
if(n<N)  
    d_a[n] = n;
```

```
}
```

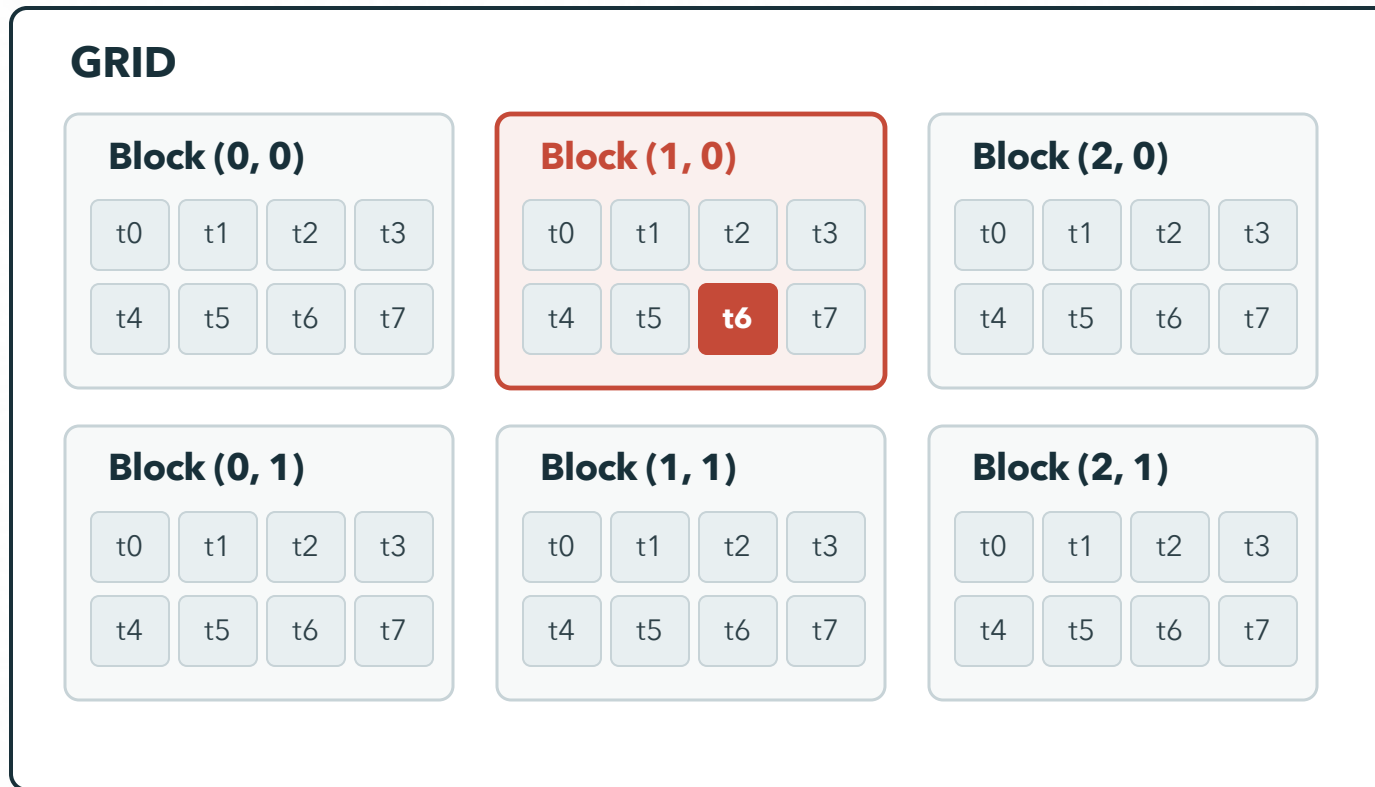
ThreadIdx & blockIdx
determine thread
rank that is mapped
to array index

Action performed by
each thread

Key observation: the loops are implicitly executed by thread parallelism
and *do not* appear in the CUDA kernel code.

Grid, block, and thread

One kernel launch creates a grid of blocks; each block contains threads.



Example: 6 blocks × 8 threads = 48. Red thread t6 maps to C[1,6].

Grid

All blocks created by one kernel launch.

Block

A group of threads that can cooperate using shared memory and a block barrier.

Thread

One execution of the kernel function, with its own local thread index.

Thread-data mapping: One-dimensional indexing

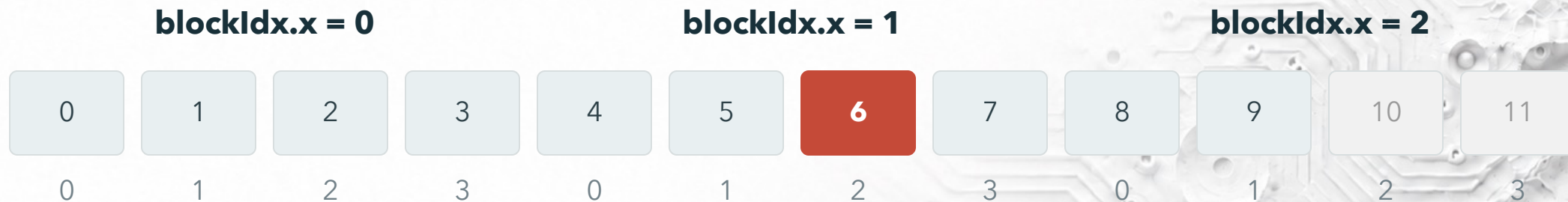
Each block covers a group of array positions. Each thread selects one position.

$$i = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}$$

Example: $\text{blockDim.x} = 4$, $\text{blockIdx.x} = 1$, $\text{threadIdx.x} = 2$

$$i = 1 \times 4 + 2 = 6$$

Output array C: N = 10 valid elements



Local thread number (threadIdx.x) repeats: 0, 1, 2, 3 in each block.

Multi-dimensional indexing

Rows and columns come from independent block and thread coordinates.

row = blockIdx.y × blockDim.y + threadIdx.y

col = blockIdx.x × blockDim.x + threadIdx.x

One thread coordinate selects one element C[row, col].

Output matrix C

0,0	0,1	0,2	0,3	0,4	0,5	0,6
1,0	1,1	1,2	1,3	1,4	1,5	1,6
2,0	2,1	2,2	2,3	2,4	2,5	2,6
3,0	3,1	3,2	3,3	3,4	3,5	3,6
4,0	4,1	4,2	4,3	4,4	4,5	4,6

```
threadIdx.x +  
threadIdx.y * blockDim.x +  
threadIdx.z * blockDim.x * blockDim.y
```

Multi-dimensional indexing

x selects a column, y selects a row, and z selects a slice of the output $C[z, y, x]$.

$$x = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}$$

$$y = \text{blockIdx.y} \times \text{blockDim.y} + \text{threadIdx.y}$$

$$z = \text{blockIdx.z} \times \text{blockDim.z} + \text{threadIdx.z}$$

Volume: $W = 5$ columns, $H = 3$ rows, $D = 3$ slices. Cell labels are (row y, column x).

Slice z = 0

	0	1	2	3	4
0	0,0	0,1	0,2	0,3	0,4
1	1,0	1,1	1,2	1,3	1,4
2	2,0	2,1	2,2	2,3	2,4

Slice z = 1

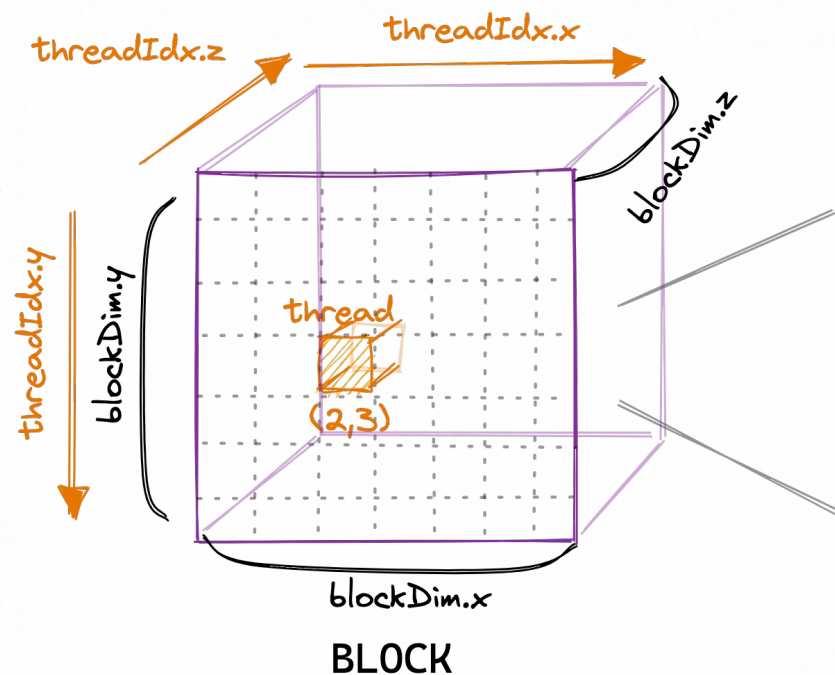
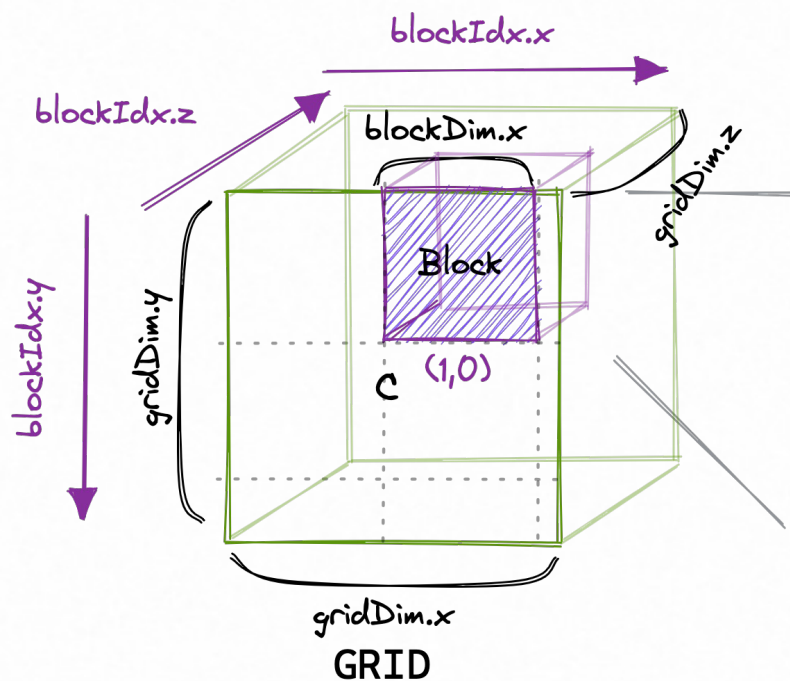
	0	1	2	3	4
0	0,0	0,1	0,2	0,3	0,4
1	1,0	1,1	1,2	1,3	1,4
2	2,0	2,1	2,2	2,3	2,4

$(x, y, z) = (2, 4, 1)$

Slice z = 2

	0	1	2	3	4
0	0,0	0,1	0,2	0,3	0,4
1	1,0	1,1	1,2	1,3	1,4
2	2,0	2,1	2,2	2,3	2,4

Thread Organization



a single thread of computation,
minding its own business

THREAD

Why do we need multi-level computation granularity?

-Tom Jerry

The GPU memory hierarchy

Smaller memories near computation support reuse, while global memory provides capacity.

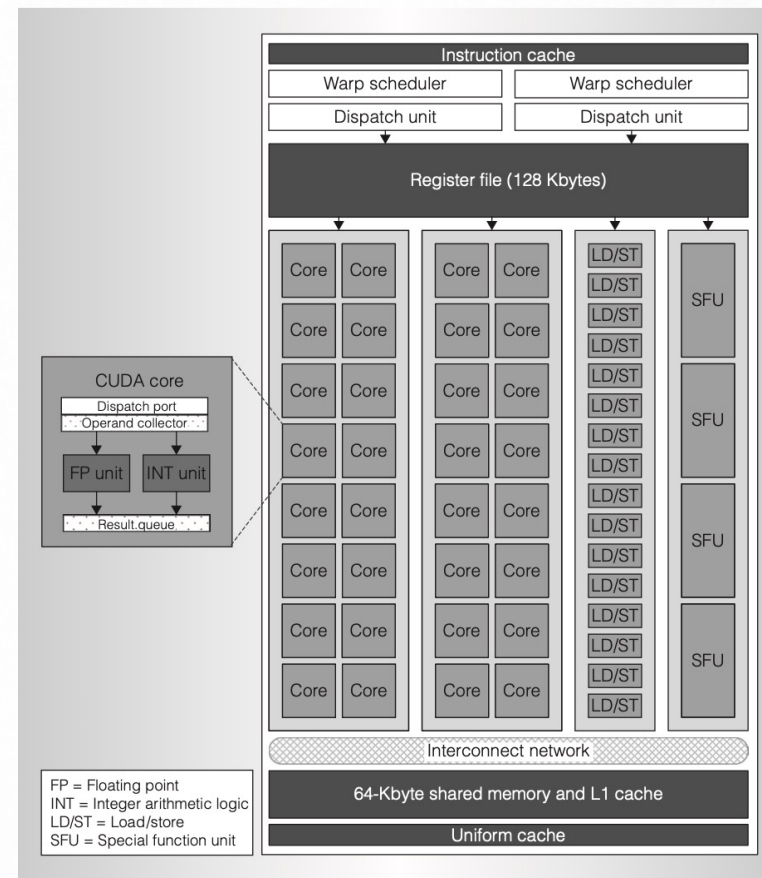
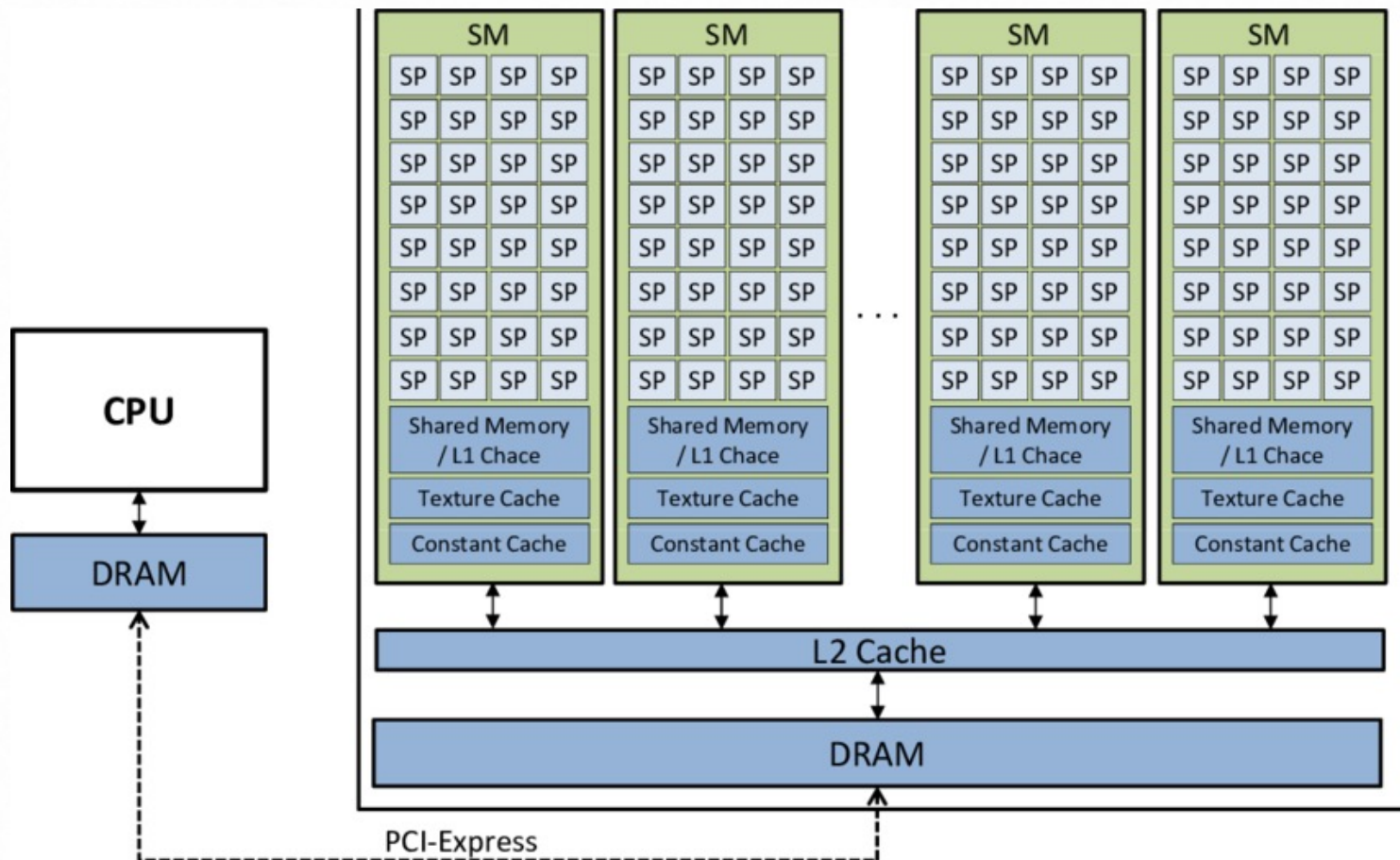
Registers One thread

Shared memory One block

L2 cache Whole GPU

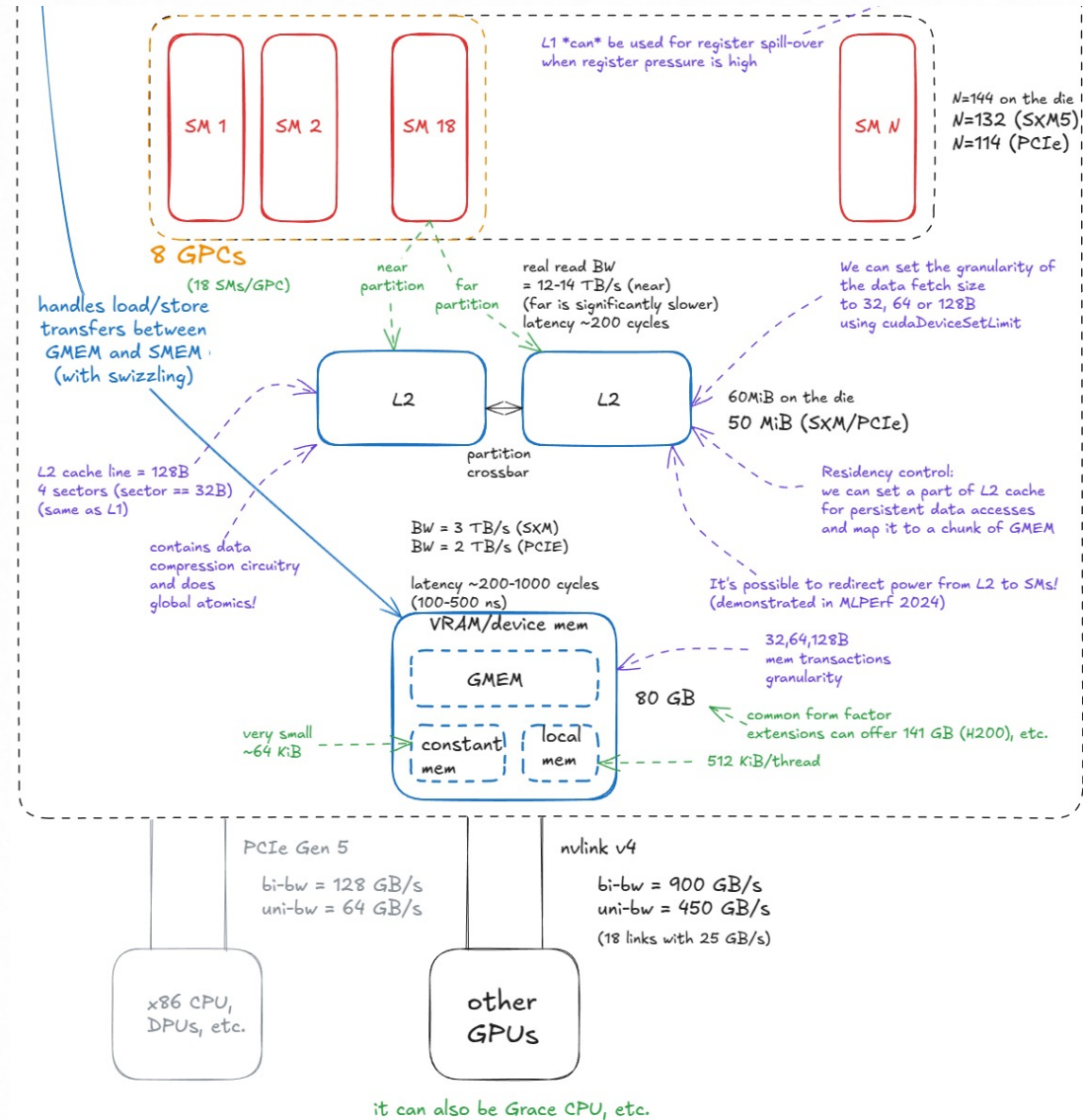
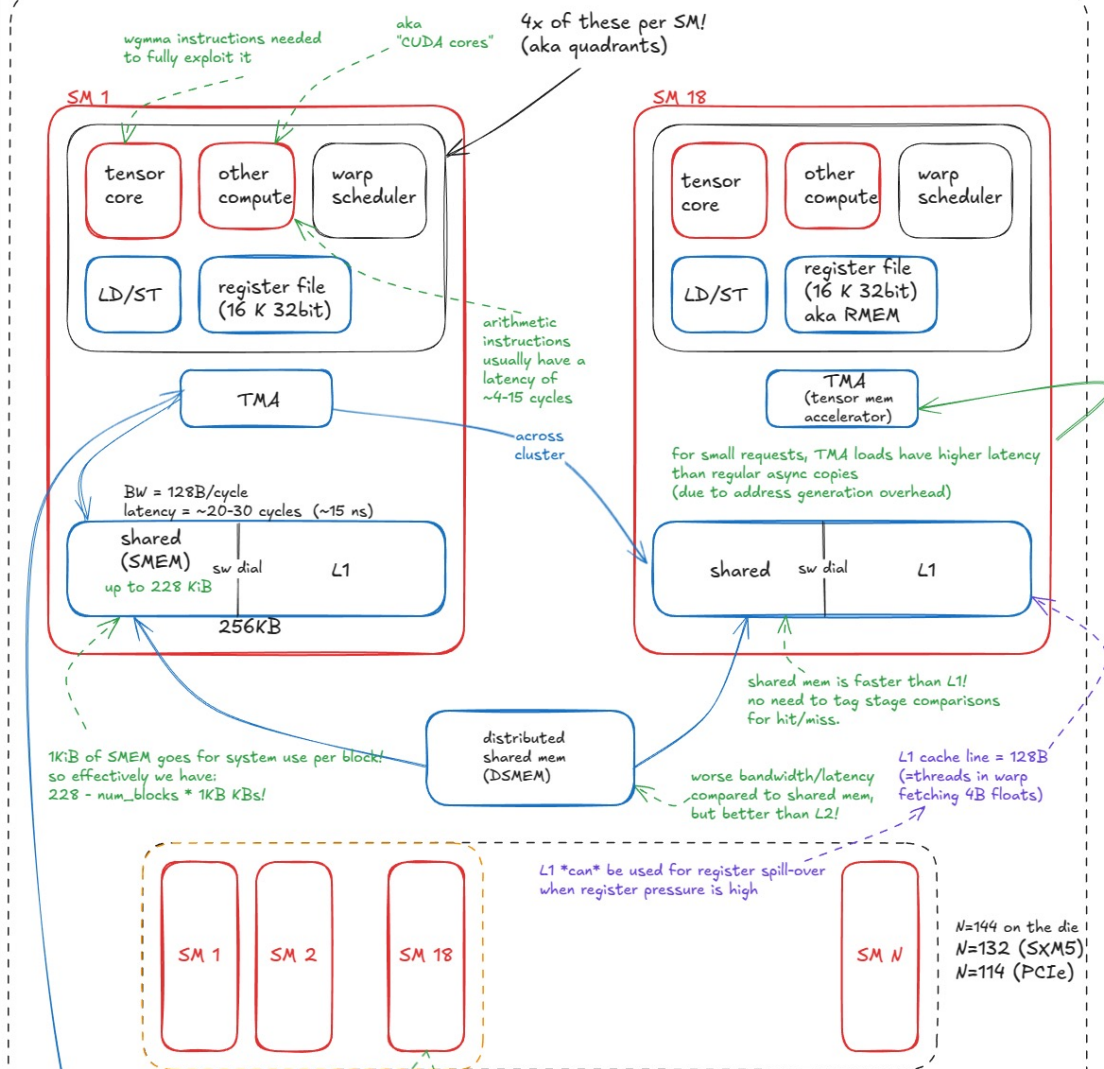
Global memory Whole GPU

Computation vs. Memory Hierarchy



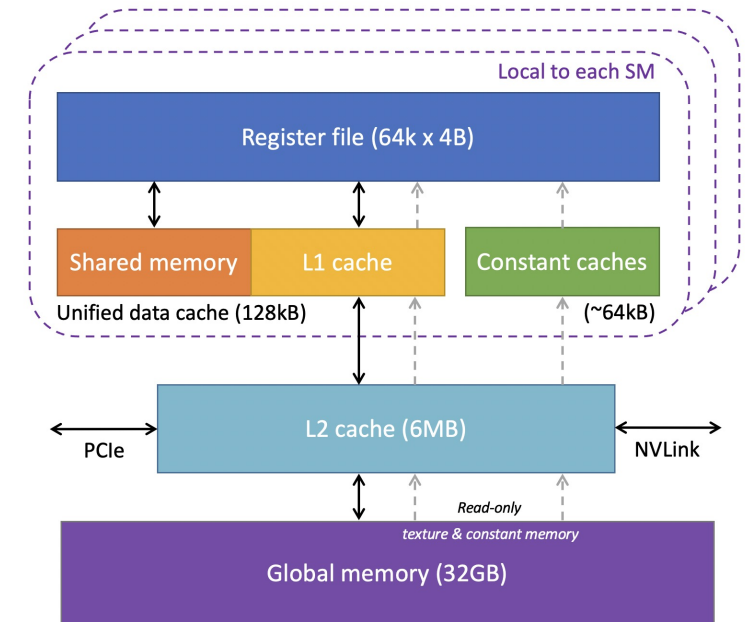
H100

power:
TDP=700W (SXM)
TPD=350W (PCIe)

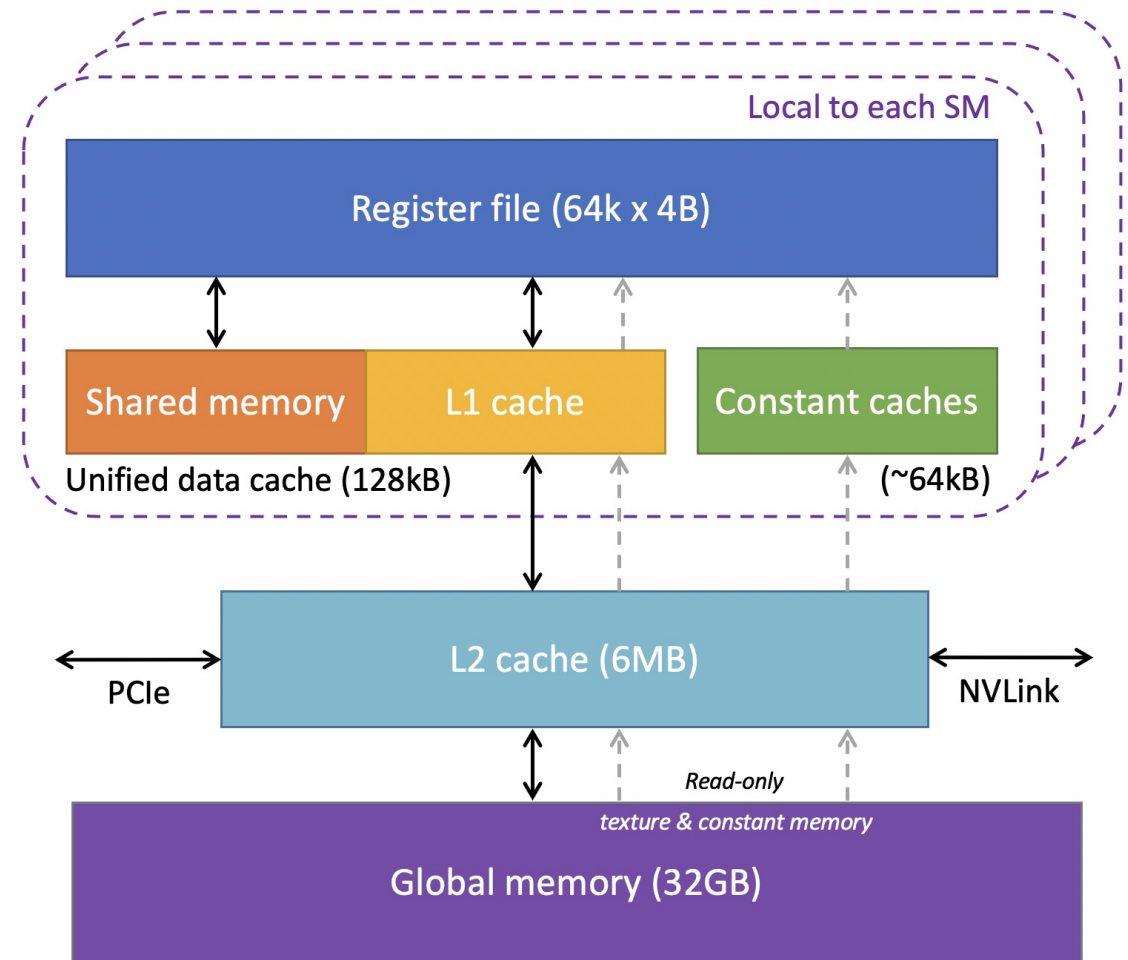
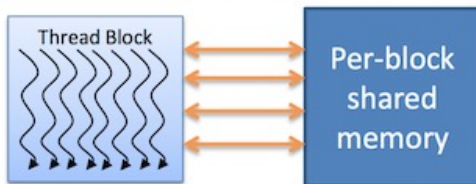
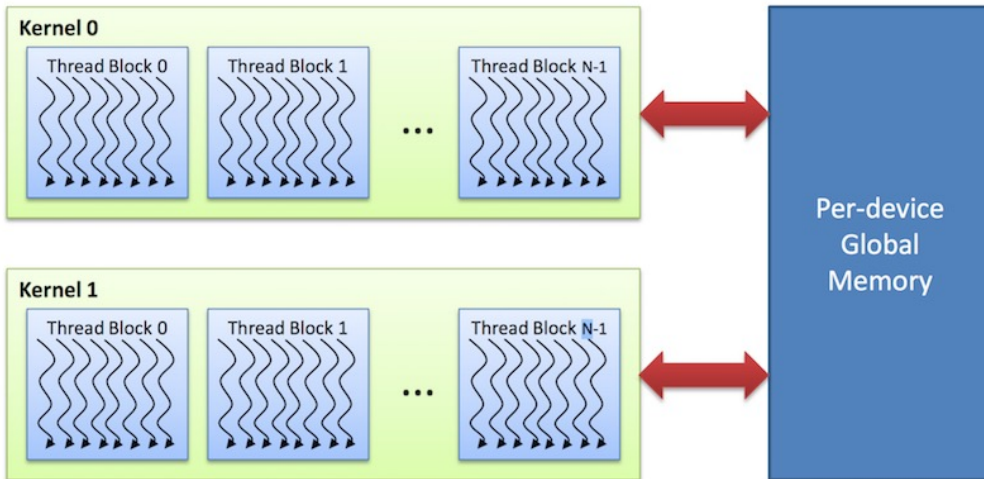


Memory Hierarchy

Level	Location	Latency	Size	Scope
Registers	SM	~1 cycle	very small	per thread
Shared Memory	SM	~20 cycles	~100 KB	per block
L2 Cache	GPU	~200 cycles	tens of MB	whole GPU
Global Memory (HBM)	GPU DRAM	~500–800 cycles	tens of GB	all threads
Host Memory	CPU	very high	large	system



Memory Hierarchy



Kernel

CUDA Concept	Computation Hardware	Memory Hierarchy
Grid	Entire GPU	Global memory (HBM / DRAM)
Block	Streaming Multiprocessor (SM)	Shared memory / L1 cache
Thread	CUDA core (warp lane)	Registers

Key intuition

- **Threads** perform computation using **registers**.
- **Threads in a block** cooperate on the same **SM** and communicate through **shared memory**.
- **Blocks in a grid** execute across the entire **GPU** and communicate via **global memory**.

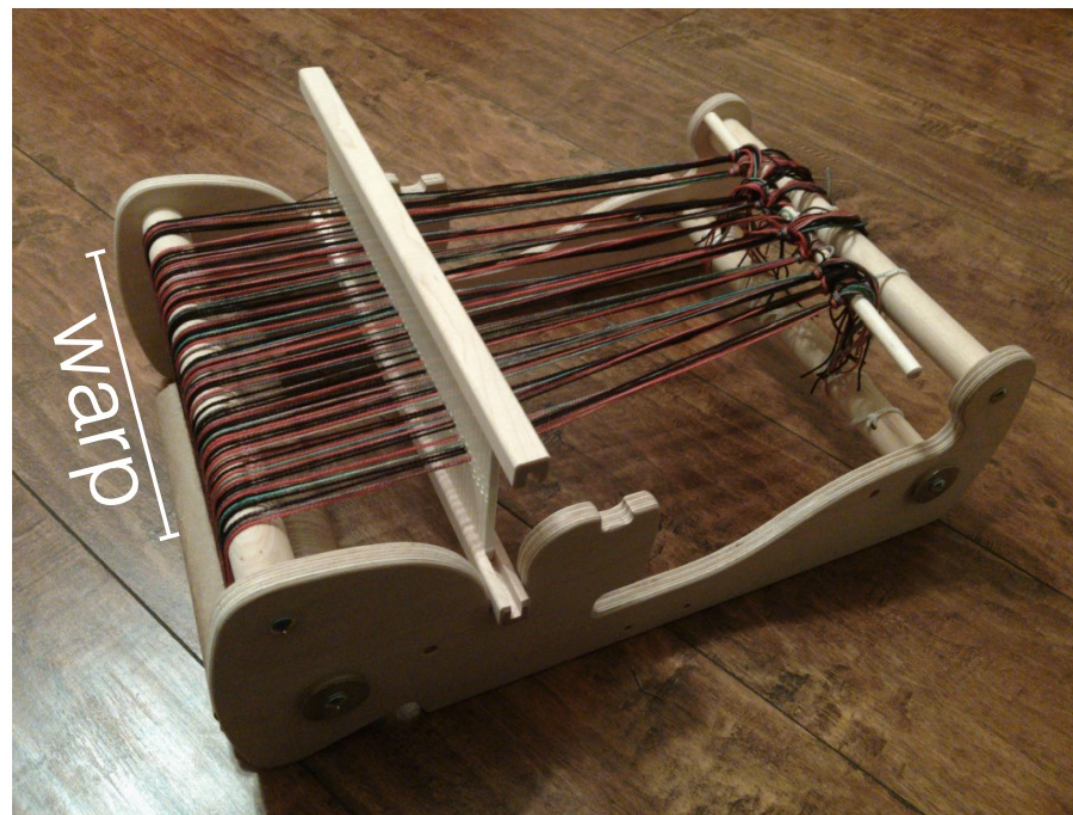
SIMD, SIMT, Warp

-Tom Jerry

Warp

In CUDA, a warp is the implementation of the Single Instruction, Multiple Threads (SIMT) execution model. A warp consists of 32 threads that execute the same instruction simultaneously, allowing for efficient parallel processing on NVIDIA GPUs.

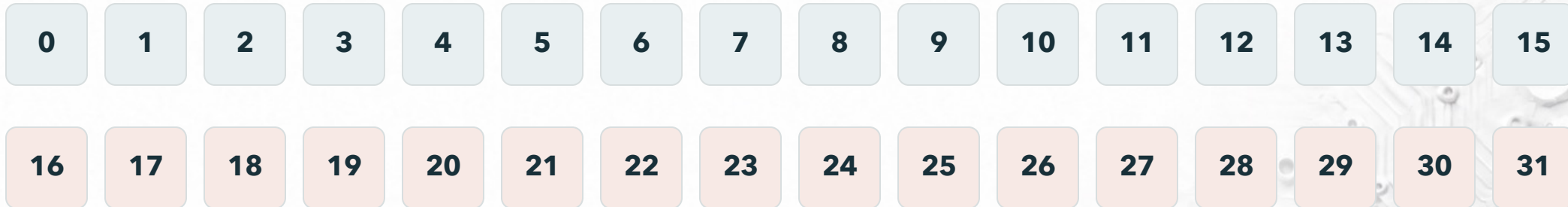
Memory Sharing: Threads within a warp can efficiently share data using shared memory, which is a fast, on-chip memory accessible to all threads in the same block.



Warps execute 32 threads together

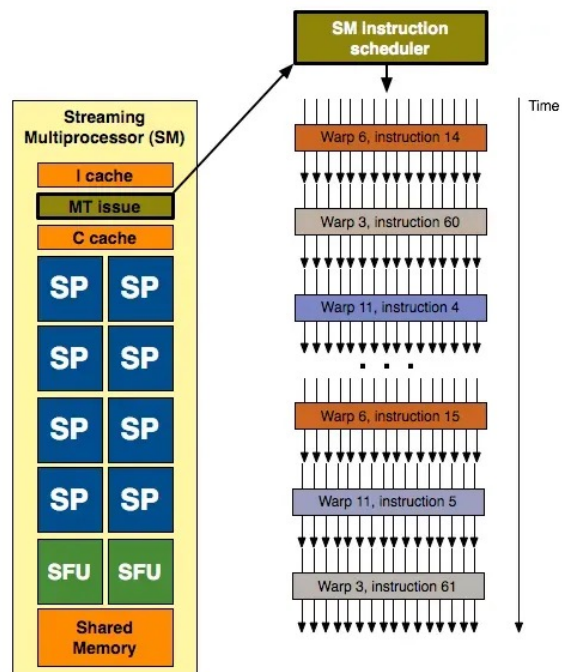
SIMT means one instruction stream controls several scalar threads.

One warp



Each thread keeps its own registers and data. The warp normally issues the same instruction for all active threads.

Warp

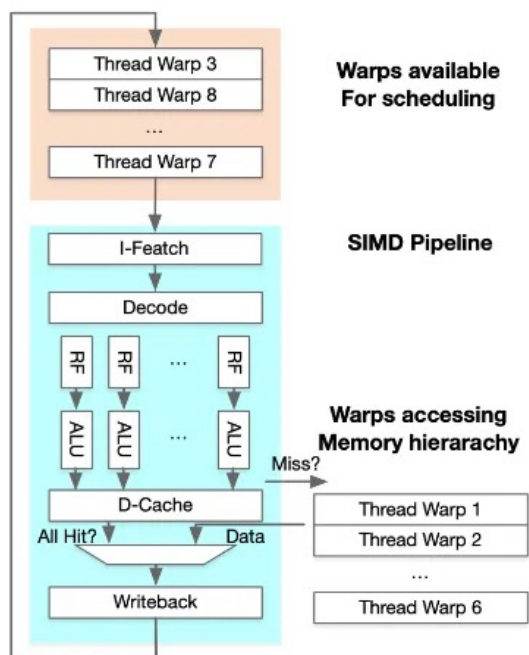


Imagine a block with 128 threads, split into 4 warps (32 threads each):

- Warp 0 might be executing a floating-point operation.
- Warp 1 might be fetching data from global memory (stalled).
- Warp 2 might be performing an integer operation.
- Warp 3 might be idle or waiting for a branch resolution.

The SM can execute instructions from Warp 0 and Warp 2 in parallel on different execution units while Warp 1 waits. This is how parallelism across warps is achieved.

Warp



Imagine a block with 128 threads, split into 4 warps (32 threads each):

- Warp 0 might be executing a floating-point operation.
- Warp 1 might be fetching data from global memory (stalled).
- Warp 2 might be performing an integer operation.
- Warp 3 might be idle or waiting for a branch resolution.

The SM can execute instructions from Warp 0 and Warp 2 in parallel on different execution units while Warp 1 waits. This is how parallelism across warps is achieved.

SIMD, SIMT, Warp

Concept	What it is	Key idea
SIMD	Hardware execution style	One instruction operates on multiple data elements
SIMT	Programming model	Many threads run the same code independently
Warp	Hardware execution unit (GPU)	32 threads executed together in lockstep

👉 **Key takeaway:** Warp is how GPUs implement SIMT on SIMD-like hardware

Branch divergence serializes paths

Threads in one warp can choose different branches, but the warp cannot issue both paths at the same time.

All threads choose one path

The warp executes one instruction sequence.
All lanes remain useful.

Threads choose different paths

The warp executes one path while masking the other lanes, then executes the second path. Useful throughput decreases for this region.

```
if (threadIdx.x < 4) {  
    A;  
    B;  
} else {  
    X;  
    Y;  
}  
Z;
```

